

Medimos hasta dónde aguanta gratis.

Pusimos una app real, cuatro microservicios en Go con conexiones en vivo, sobre la **capa gratuita de Render** y le echamos encima hasta mil usuarios simulados a la vez. Esto es lo que encontramos, para que no lo descubras tú en producción un domingo.

100

usuarios a la vez, cero errores y alrededor del 10 % de la memoria en uso. La respuesta, eso sí, ya iba al límite

1 de 4

conexiones en vivo no logran conectarse al llegar a mil usuarios simultáneos

46.6 %

de la memoria usada al momento de fallar: le sobraba más de la mitad

Esperábamos que se muriera de falta de memoria. Se murió de acoplamiento.

Composición de sesiones WS a 1 000 VUs (rep. 1)

Completadas	3 528 (76.1 %)
Caídas	1 106 (23.9 %)

RAM pico por servicio: 100 → 1 000 VUs

Servicio	RAM pico (MB)
api-gateway	238 MB
realtime	231 MB
groups	55 MB
habits	22 MB

Lo que se rompe:

de cada cuatro personas que abren la app a la vez, una pierde la conexión en vivo.

Lo que sobra:

el espacio a la derecha de los puntos es memoria sin usar. El límite nunca se tocó.

¿Por qué, entonces?

Cada vez que alguien abre una conexión en vivo, ese servicio le pregunta a otro «¿esta persona pertenece a este grupo?». Correcto en seguridad, pero amarra el tiempo real a la velocidad de la base de datos: cuando esa consulta se atora las conexiones caen, y cada reintento vuelve a pegar sobre el mismo servicio saturado. El cuello de botella estuvo en cómo amarramos las piezas.

Lo que de verdad te va a doler

Te suspenden por datos, no por CPU

La cuota es de 5 GB al mes por cuenta. Nuestras pruebas movieron 17.8 GB y suspendieron dos. Comprimir bajó el tráfico 89.6 %.

El servicio se duerme a los 15 minutos

Despertar tarda de 10 a 14 segundos por servicio. Encadenados, el usuario ve la app «caída» entre 30 y 60 segundos.

La base de datos también se pausa

Tras 8 días sin actividad la nuestra se pausó sola: los servicios arrancaban bien y fallaban en la primera consulta.

Rinde distinto según cómo la usaste antes

La base gratuita acumula créditos de procesador en reposo y los gasta bajo carga. Medimos la firma: memoria al tope y procesador ocioso. Que la causa fueran los créditos es lo más probable, no algo que confirmáramos.

Qué haríamos distinto desde el día uno

1

Comprimir las respuestas

Lo más barato y lo único que medimos directo: 89.6 % menos datos en el cable.

2

No consultar la base en cada conexión

Cachear el permiso o firmarlo en un token rompe la cadena que nos tumbó.

3

Contar con el arranque en frío

Medio minuto en blanco duele más que 200 ms de lentitud. Avisa o despierta antes.

LETRA CHICA: LO QUE SÍ MEDIMOS Y LO QUE NO

- Medido: 100 y 1000 usuarios simultáneos, tres repeticiones cada uno.
- Es un caso, no una ley: Go + Render + Supabase. Otra combinación puede romperse en otro lado.

- No probado: planeábamos 2500 y 5000, pero las reglas del plan gratuito agotaron nuestra ventana de pruebas.
- Sin datos de personas: toda la carga fue simulada contra una cuenta de prueba; no participaron usuarios reales.

- Estimado, no medido: ~3200 usuarios activos al día. Sale de un cálculo con supuestos declarados.
- Sesgo conocido: los usuarios simulados compartían cuenta, así que la base de datos la tuvo más fácil que en la vida real.

dydi · Equipo 3 · Integradora 2026 · Universidad Tecnológica de Durango

García Páez David Israel · Solís Flores Irvin Alfonso · Cervantes Guerrero Keila Yuridia · Casiano Gamzi Juan David

El artículo completo, el arnés de pruebas y los datos crudos de las once corridas están publicados:

github.com/davidgarcia57/Dydi